IDATG2208 - Mandatory Assignment - 3

Deadline for submission - 21 Oct, 2026

Instructions

- Set random seeds to 42 for all experiments (`tf.keras.utils.set_random_seed(42)`).
- Use only the specified dataset subset for reproducibility. Do not shuffle the dataset before taking the subset.
- For datasets loaded via TensorFlow Datasets (`tfds`), use `shuffle_files=False` and the exact split strings given.
- Report your answers as percent numbers for test accuracy. The code and plots depicting training and validation loss should be provided along with the test accuracy.
- Unless otherwise stated, use `validation_split = 0.1` during training.

Exercise-1: Training Deep Neural Network on Fashion-MNIST [15 points]

Train a controlled deep neural network on the Fashion-MNIST dataset (`tf.keras.datasets.fashion_mnist`). Set random seeds to 42. Load Fashion-MNIST and normalize pixel values to [0,1]. Build the network using the following configuration:

- Flatten input images to $28 \times 28 = 784$ features
- 3 hidden layers, 100 neurons each
- ELU activation function
- He normal initialization
- Output layer: 10 neurons with softmax
- Optimizer: Nadam
- $learning_rate = 0.001$, loss=`sparse_categorical_crossentropy`
- EarlyStopping callback: monitor validation loss, $patience = 5$, restore best weights
- $epochs = 50$, $batch_size = 32$
- Use only the first 2000 training samples and first 400 test samples

Q1.1 Report the obtained test accuracy.

Exercise-2: Training Deep Neural Network on CIFAR-100 (Coarse Labels) [15 points]

Train a controlled deep neural network on the CIFAR-100 dataset using its 20 coarse (superclass) labels, loaded with `tf.keras.datasets.cifar100.load_data(label_mode='coarse')`. Set random seeds to 42. Load CIFAR-100 and normalize pixel values to [0,1]. Build the network using the following configuration:

- Flatten input images to $32 \times 32 \times 3 = 3072$ features
- 4 hidden layers, 256 neurons each
- ELU activation function
- He normal initialization
- Output layer: 20 neurons with softmax
- Optimizer: Nadam
- $learning_rate = 0.001$, loss=`sparse_categorical_crossentropy`
- EarlyStopping callback: monitor validation loss, $patience = 5$, restore best weights
- $epochs = 50$, $batch_size = 128$
- Use only the first 5000 training samples and first 1000 test samples

Q2.1 Report the obtained test accuracy.

Exercise-3: Regularization with Alpha Dropout and MC Dropout [15 points]

Using the Fashion-MNIST dataset, extend the network from Exercise 1 by applying Alpha Dropout. Then, without retraining, use Monte Carlo (MC) Dropout at inference to estimate if you can achieve better accuracy. Set random seeds to 42. Use the following configuration:

- Flatten input images to $28 \times 28 = 784$ features
- Standardize inputs (zero mean, unit variance, using training set statistics)
- 3 hidden layers, 100 neurons each
- SELU activation function (required for Alpha Dropout)
- LeCun normal initialization
- Alpha Dropout rate: 0.1 in all hidden layers
- Output layer: 10 neurons with softmax

- Optimizer: Nadam
- *learning_rate* = 0.001, loss=`sparse_categorical_crossentropy`
- EarlyStopping callback: monitor validation loss, *patience* = 5, restore best weights
- *epochs* = 50, *batch_size* = 32
- Use only the first 2000 training samples and first 400 test samples
- For MC Dropout, enable dropout during inference (`model(X, training=True)`) and average predictions over 20 stochastic forward passes

Q3.1 Report the test accuracy of the network with Alpha Dropout applied during training.

Q3.2 Report the MC Dropout-enhanced accuracy (averaging 20 stochastic predictions).

Exercise-4: Transfer Learning with Pre-trained CNN on EuroSAT [15 points]

Use a pre-trained convolutional neural network (CNN) as a feature extractor and train a classifier on a subset of the EuroSAT satellite image dataset (10 land-use classes, RGB images of size 64 × 64). Set random seeds to 42. Follow the configuration below:

- Load EuroSAT with `tfds.load('eurosat/rgb', split=['train[:2000]', 'train[2000:2500]'], as_supervised=True, shuffle_files=False)`. The first split is the training set (2000 samples) and the second split is the test set (500 samples)
- Normalize pixel values to [0,1]
- Load `MobileNetV2` from `tensorflow.keras.applications`, with `include_top=False`, `weights='imagenet'` and `input_shape=(64, 64, 3)`
- Freeze all layers of the pre-trained base
- Add a classifier on top:
  - GlobalAveragePooling2D
  - Dense layer with 128 neurons, ReLU activation
  - Dropout: 0.2
  - Output layer: 10 neurons with softmax
- Optimizer: Adam, *learning_rate* = 0.001
- Loss: `sparse_categorical_crossentropy`
- *epochs* = 5, *batch_size* = 32

Q4.1 Report the test accuracy of the model.

Exercise-5: Deeper CNN Training on KMNIST [20 points]

Train a controlled deep convolutional neural network (CNN) on a subset of the Kuzushiji-MNIST (KMNIST) dataset of handwritten Japanese characters (10 classes). Set random seeds to 42. Build the network using the following configuration:

- Load KMNIST with `tfds.load('kmnist', split=['train[:3000]', 'test[:500]'], as_supervised=True, shuffle_files=False)` and normalize pixel values to [0,1]
- Use only the first 3000 training samples and first 500 test samples
- Input shape: $28 \times 28 \times 1$
- CNN architecture:
  - Conv2D: 32 filters, 3×3 kernel, ReLU activation
  - Conv2D: 32 filters, 3×3 kernel, ReLU activation
  - MaxPooling2D: 2×2
  - Conv2D: 64 filters, 3×3 kernel, ReLU activation
  - Conv2D: 64 filters, 3×3 kernel, ReLU activation
  - MaxPooling2D: 2×2
  - Flatten
  - Dense: 256 neurons, ReLU activation
  - Dropout: 0.3
  - Output layer: 10 neurons with softmax
- Optimizer: Adam, $learning_rate = 0.001$
- Loss: `sparse_categorical_crossentropy`
- $epochs = 15$, $batch_size = 32$

Q5.1 Report the test accuracy of the deeper CNN model.

Exercise-6: CNN with SGD, MC Dropout, and Epistemic Uncertainty on KMNIST [20 points]

Train a controlled convolutional neural network (CNN) on a subset of the KMNIST dataset using the SGD optimizer. Then, apply Monte Carlo (MC) Dropout at inference to estimate both test accuracy and epistemic uncertainty. Set random seeds to 42. Use the following configuration:

- Load KMNIST exactly as in Exercise 5 and normalize pixel values to [0,1]
- Use only the first 3000 training samples and first 500 test samples

- Input shape: $28 \times 28 \times 1$
- CNN architecture:
  - Conv2D: 32 filters, 3×3 kernel, ReLU activation
  - MaxPooling2D: 2×2
  - Conv2D: 64 filters, 3×3 kernel, ReLU activation
  - MaxPooling2D: 2×2
  - Flatten
  - Dense: 128 neurons, ReLU activation
  - Dropout: 0.25 (keep during inference for MC Dropout)
  - Output layer: 10 neurons with softmax
- Optimizer: SGD with momentum = 0.9, $learning_rate = 0.01$
- Loss: `sparse_categorical_crossentropy`
- $epochs = 15$, $batch_size = 32$
- For MC Dropout:
  - Enable dropout during inference
  - Average predictions over 20 stochastic forward passes
  - Compute the epistemic uncertainty as the predictive variance across passes: for each test sample, compute the variance of each class probability over the 20 passes, then average over the 10 classes

Q6.1 Report the plain test accuracy of the CNN trained with SGD (no MC Dropout).

Q6.2 Report the MC Dropout-enhanced accuracy (averaging 20 stochastic predictions).

Q6.3 Compute the average epistemic uncertainty (mean predictive variance) across all test samples. Report it as a deterministic number rounded to 3 decimal places.